ENSC 413 Final Report: Real-Time Audio Denoising ML System Design

Mingyang Cheng 301472892
Hyun (Justin) Yang 301461361

1. Project Description / Preliminary Research

The objective of the project was to implement a real-time noise filtering system utilizing machine learning. While there are many pre-existing designs that target this area of study, the focus of this project was to create a light-weight version that produces reasonable results by the end of the term. Another objective with this project was to gain a deeper understanding of how machine learning is able to process information through a hands-on project and learn to adapt these skills to future projects.

In today's society, there are many companies and applications that utilize machine learning to reduce and/or remove the noise of a live audio file or pre-recorded files. Common examples of these denoising systems include Microsoft Team's audio filter during live calls [1] and Discord's Krisp during calls with friends [2]. Although these applications are widely used, as their designs are proprietary to the company, there were not many details on specific system design other than comments such as "using AI" [1] or "using a deep neural network" [3]. Besides the two, there are also many open source projects such as the GitHub user immohann that designed a 2 layer convolutional neural network (CNN) to filter out the noise [4]. However, it is important to note that these designs are focused on denoising audio files that were prerecorded, rather than looking to denoise audio files in a real-time application. When observing various other designs that both did and did not use machine learning, one of the core methods for implementation was using short-time Fourier transform (STFT) for changing the audio files into spectrograms [5] [6]. With this knowledge, rather than attempting to redesign, the group's objective was to experiment and see the possibilities of different networks within the given time frame to test new possible network designs. This ultimately resulted in the design and testing of two designs: Multi-Label Classification and U-Net based STFT masking.

2. Planning

The group's attention was first divided into two areas: understanding the current designs available and determining how different designs could be explored. The process began with looking deeper into the designs mentioned in the previous section, as well as using resources learned during the term to apply new changes to the designs.

This led to the decision to develop two separate approaches in parallel: a multi-label classification system and a STFT based masking system. The two designs were first made as prototypes to determine the plausibility of both designs and further iterative changes were made to work on experimenting and comparing the results. The following sections will highlight the core designs of each prototype and the results from each implementation.

3. Datasets

This section will go in-depth about the selection of audio files and utilizing augmentation to create a wider variety of denoises.

3.1 Dataset Selection / Variation

The core of the dataset was mainly found online through openly available resources. The majority of speech audio was procured through a GitHub provided by Microsoft with thousands of speech audio clips [7] and through Research Google's Audioset [8]. The noise files were sourced through various different websites such as Kaggle for eating noises [9], GitHub repository by karolpiczak [10] for keyboard clips and coughing, and Zenodo for shorter keyboard clicks [11]**.** The following table shows the spread of data used for the training of the models:

Category of Noise	**No. of samples**
Speech [7][8]	10000
Keyboards [10][11]	40
Munching [9]	35
Coughing [10]	30

Table 1: List of resources found for project

While the datasets contained a high quantity of speech files, specific noises were harder to find as noise datasets typically featured extra noise like air conditioning or fans that were not included in the project's objective. To widen the variety of audio files, data augmentation was used.

3.2 Dataset Augmentation / Creation

As mentioned, the noise files lacked enough variety for the model to generalize well for audio clips it has never seen. To combat this problem, the noise dataset was processed through a Python script to save 5 new copies per 1 original noise file with the following possible operations applied: pitch shifting, time stretching and changing the volume of the audio files. For added randomness to the dataset, the audio files were not all augmented the same way, rather, the audio effects were chosen through chance. For example, each time a new copy is being made, there is a 70% chance of the audio file being time stretched or a 50% of the audio file having its volume changed. These new audio files were then mixed with the original speech files at random with STFT further tuning during training to use the signal-noise ratio for increasing or decreasing noise audio.

4. Classification-Based Audio Attenuation

The first approach uses a simple classification model that was taught in class and was familiar to the group. It was chosen as a fail-safe option if the more common STFT-based methods could not be implemented correctly for the project. At the time, it was assumed that detecting the type of noise in an audio segment would be enough to reduce its effect. However, during implementation, it became clear that classification alone cannot separate noise from speech. Instead, the model can only detect whether certain types of noises are present, and this information must then be used to attenuate the overall audio rather than selectively remove only the noise.

This approach is therefore referred to as Classification-Based Audio Attenuation [12]. This system uses a multi-label classifier to predict the presence of one or more noise types within an input audio segment. These predictions are then used to control how much the signal is attenuated during playback. Since the model does not perform source separation, it cannot isolate speech from background noise at the sample or frequency-bin level. As a result, any attenuation applied by the system affects the full audio segment, including both the unwanted noise and the desired speech.

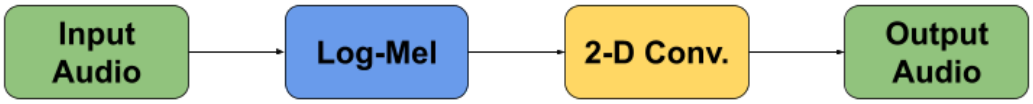

Figure 1: Design of multi-label classification system

4.1 Mutli-label Classification

Multi-label classification is a learning task in which one input sample can belong to multiple classes at the same time. This is suitable for the project because real audio segments may contain more than one sound source simultaneously, such as speech together with keyboard noise or coughing [13]. Therefore, instead of assigning only one label to each clip, the system is designed to predict all relevant noise classes present in the segment, and have different attenuation strength for each sound segment.

4.2 Dataset Processing and Labeling

Due to the nature of the multi-label classification model, datasets required both manual labelling and audio mixing for the model to learn meaningful combinations of sounds. The dataset was organized so that labels could be assigned as binary combinations across the noise classes and clean audio. In the preprocessing pipeline, labels were inferred from the folder names, allowing single-source clips such as *keyboard* as well as mixed clips such as *speech+keyboard* or *munching+cough* to be converted into multi-hot target vectors for training.

Each audio file was converted into a normalized log-mel spectrogram before being passed to the model. The log-mel spectrogram has the advantage of being a more compact and perceptually meaningful representation of the audio, placing more emphasis on lower-frequency structure while compressing less informative high-frequency detail. Using log-mel also reduces the dynamic range of the signal, making quieter components easier for the model to learn, and making the model significantly faster to train.

The preprocessing code uses 2-second clips sampled at 16kHz and generates training clips at multiple lengths, including 2.0s, 1.0s, 0.5s, and 0.25s. Although the waveform duration varies, each clip is converted into a log-mel spectrogram with a fixed final time dimension so that all inputs share the same shape during training, testing, attenuation, and real-time application. This allows a single model to learn from both longer contextual segments and shorter transient events. However, preparing the dataset is labour-intensive, since the model performance depended heavily on whether the assigned labels accurately reflected the sounds present in each segment. In addition, mixed samples had to be prepared carefully so that the classifier could learn overlapping classes rather than only isolated examples of each sound.

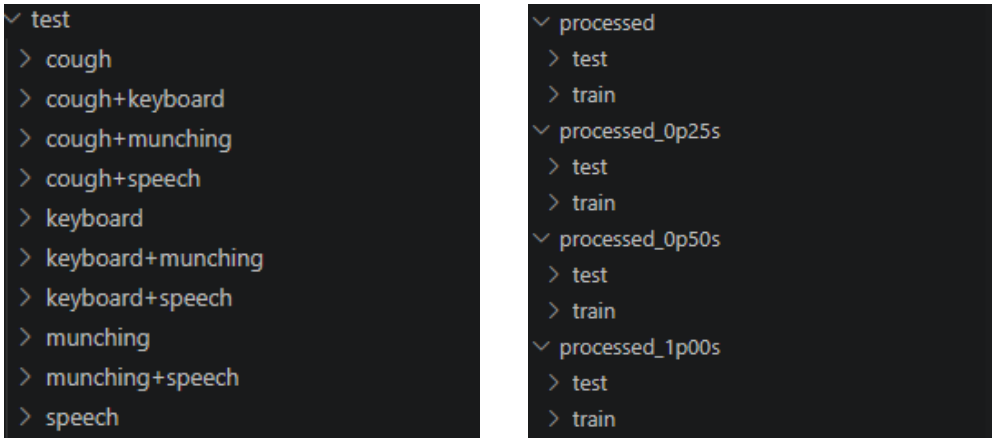

Figure 2: File structure of audio files with labeling

4.3 Model Architecture

The model architecture was designed as a lightweight 2D convolutional neural network operating on log-mel spectrogram inputs rather than raw audio waveforms. Each audio clip is first converted into a normalized spectrogram with fixed time dimensions, allowing it to be treated similarly to an image for feature extraction. The network begins with batch normalization, followed by four convolutional layers with 16, 32, 64, and 128 filters all using the activation function ReLU, with the first three convolution stages each being followed by max-pooling to gradually reduce spatial size while preserving important patterns. After the final convolution layer, global average pooling is used to compress the learned feature maps, followed by a dense layer with 64 units and a dropout layer for regularization. The output layer uses sigmoid activation so that each class can be predicted independently, which is appropriate for the multi-label nature of the task. During training, the model is optimized using Adam and weighted binary cross-entropy [14] is used as the loss function to handle the imbalances in sample size.

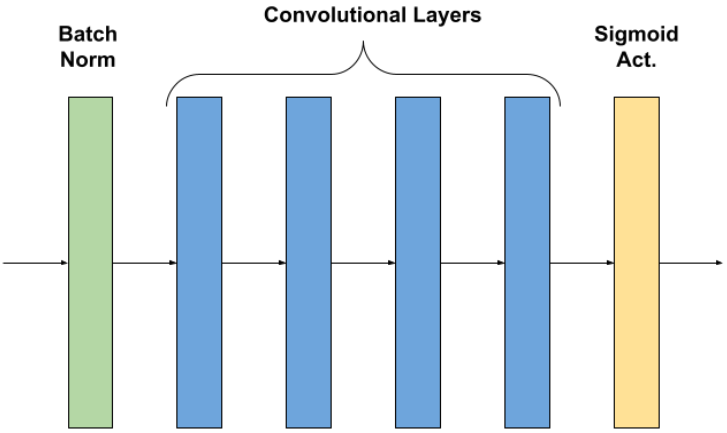

Figure 3: Design of 2D Convolutional Neural Network design

4.4 Attenuation Strategy

Designing the attenuation algorithm proved more difficult than building the classifier itself. Rather than performing true source separations, the system uses the output probabilities of a multi-label classifier to control a time-varying gain applied directly to the input waveform. In the attenuation stage, predictions from different audio windows sizes, including 2.0s, 1.0s, 0.5s, and 0.25s, are combined to estimate how likely each noise class is to be present at a given moment. These class probabilities are then compared against confidence thresholds and converted into gain values, where stronger confidence in unwanted noise results in stronger attenuation. The final gain is applied to the full audio signal over time, reducing the impact of detected noise events.

4.5 Real-time Implementation

The real-time application of the prototype was able to classify short audio segments reasonably well, but it was less successful at producing smooth transitions between segments containing unwanted noise and segments containing mainly speech. In real-time use, a small playback delay of 0.5s is implemented so that enough recent audio can be buffered before suppression is applied, allowing the model to consider short-term context rather than reacting only to the current block. The implementation uses the same multi-label, classification-guided attenuation strategy described above, but only with a window of 0.5s and 0.25s to achieve “real-time”. Although this improves the responsiveness of the system, it also introduces a tradeoff between accuracy and latency, since shorter windows tend to perform worse at detecting noise given the model architecture.

5. STFT-based Masking

Compared to the use of classification for design 1, the design of STFT-based masking achieves its audio denoising by utilizing a neural network to create a mask for the input audio file. The general dataflow is to first transform the audio file into time-frequency domain and detect an applicable mask for the noise of the file. This is then multiplied to the audio file and reproduced as an output post inverse STFT. The next sections will highlight each portion of the design in detail.

Figure 4: Design of STFT-based masking system

The STFT based design was chosen as the group's better final design candidate as it showed more promising results during prototype testing and the ease of model architecture changing allowed a more iterative approach in terms of development.

5.1 STFT and ISTFT

Rather than utilizing the log-mel method found in the multi-label method, this design utilized STFT and ISTFT to transform the input audio files of amplitude and time based audio files to time-frequency spectrograms [15]. The STFT design offers benefits as it allows for the waveforms to be separated into appropriate frequency bins which the model is able to extract the characteristics of speech and noise. For example, the humming of a fan may result in a hard-to-extract audio 1D waveform but transforming using STFT allows the specific frequency of the fan's hum to be detected and masked. There are also benefits in terms of the transform retaining phase information which can be used post model processing to allow the recreation of the audio file with the new mask and the phase data present. As of now, the model still relies on the noisy audio files phase but this is planned to be changed in future development.

$$X[n,\omega) = \sum_{m=-\infty}^{\infty} w[n-m]x[m]e^{-j\omega m}$$

Equation 1: Discrete Short-Time Fourier Transform [15]

$$x[n] = \frac{1}{2\pi w[0]} \int_{-\pi}^{\pi} X[n,\omega)e^{j\omega n} d\omega$$

Equation 2: Discrete Inverse Short-Time Fourier Transform [15]

5.2 Model Architecture

The first implementation of the STFT model was based on a convolutional neural network similar to the designs of immohann [4] and a MatLab denoising example using machine learning [16]. This was done during the initial prototyping phase to use as a possible basis for a better understanding of the

project requirements. This design was soon abandoned to try newer implementations of neural networks and determine its effectiveness. The most recent model is a U-Net based architecture inspired by a research paper attempting to denoise using a similar architecture [17], utilizing 4 encoding layers, a bottleneck, 4 decoding layers and a 1 channel spectrogram reconstruction layer. The main difference between the paper and the objective of this project was the use of real-time. The benefits of a deeper model is the higher denoising accuracy when inputting audio files but is problematic when the deeper model may cause an increase in total processing time. To combat this problem, 2 encoder layers and 2 decoder layers with a skip connection were implemented instead to reduce the amount of time required to process information. All layers were 2-D convolutions with the first three layers including batch normalization and ReLU, while the final layer featured a sigmoid function to determine the required mask. Additionally, while the paper's design focus was to create a clean spectrogram through training, the project's objective was to generate an applicable mask that could tune down/remove the noise heard in noisy audio files. Besides these changes, several other modifications were made to suit the objective of the project and experiment with varying design parameters outside of the model design..

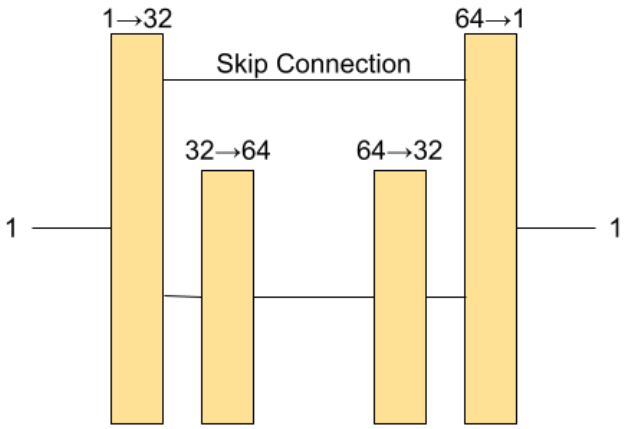

Figure 5: Simplified image of U-Net based structure used for STFT model

5.3 Differences

Besides the architecture remodelling mentioned in section 5.2, minor changes were also made to the parameters for the model itself. Namely, the optimizer was changed from utilizing Adam to using AdamW, the loss function was changed from Mean Square Error (MSE) to L1 based with a noise strength factor to enhance quieter sounds and reduce louder sounds, and the training setup also varied as different target noise clips were used during training [17]. Other features like a learning rate scheduler and early stop condition were added during the training phase to help but the core design of the system remained similar to the original. The objective of the re-design was to learn through trial and error the benefits and negatives of changing small parameters rather than reinventing an already pre-established design.

5.4 Real-Time Implementation

The system design of the real-time component followed a similar principle to the multi-label design. The main structure utilized Python libraries, such as soundcard [18] and scipy [19], to account for microphone recording and saving as .wav files. The script is currently designed to display the latency of the audio playback/recording when no model is being applied (unmasked) and when the model is being applied (masked). The details of latency will be consulted in a later section of the report.

6. Design Differences and Challenges

The main differences between the two designs is in regards to the core concept of how denoising is achieved. While the multi-label system is designed to detect noise as a certain class type and attenuate the volume accordingly, the masking system allows for specific noise frequencies common in certain noises to be removed rather than specific noise detection. This also came with an array of problems in regards to how well the final system worked.

For the multi-label classification design, the main advantage is simplicity. The structure was easier to understand, train and adapt from concepts already taught in class. It allowed the group to build a working prototype relatively quickly and test how machine learning could be integrated into a real-time audio application. The main limitation came to be that classification alone cannot separate noise from speech. Instead, it predicts whether certain sounds are present in a segment, and the system must then attenuate the full waveform based on that prediction. As a result, when unwanted noise and speech are present at the same time, both are reduced altogether. This reduces the system's overall precision and can lead to unnatural transitions between noisy and non-noisy segments, especially in real-time use. As such, the main challenge of the system lies not with the model itself, but with the attenuation strategy: combining confidence levels, short transient windows, longer contextual windows, and varying attenuation strengths into a single algorithm that produces smooth, consistent, and natural overall volume control. It is currently a proof of concept, rather than a working design.

The STFT-based masking design approaches the problem in a more direct way, by working in the time-frequency domain. Rather than deciding only what class of noise is present, it attempts to estimate which parts of the spectrogram should be preserved and which parts should be suppressed. In theory, this provides a more targeted form of denoising, since the model can reduce unwanted components without needing to lower the entire signal. However, this method introduced greater complexity in every stage of development. The model's preprocessing and postprocessing steps were more sensitive, and the final audio quality depended heavily on how accurately the mask was estimated. Additionally, because the signal had to be transformed and reconstructed, the design was more difficult to integrate into a real-time pipeline with low latency.

Data quality is one of the major challenges for both cases. In the case of the classification model, it learns to associate overall patterns in an audio segment with the target classes, and so unclear or inconsistent labels can easily confuse training. For example, some noises can share very similar short, high-energy characteristics. A sharp keyboard click and a short scream, for instance, may both appear as sudden broadband events in a spectrogram, especially when analyzed over a short window. Additionally,

if a clip is labeled as containing keyboard noise but the actual keyboard sound only occurs briefly in one part of the segment, then the label may not accurately describe the whole window, further reducing the reliability of the model. For the masking-based design, the system requires accurate mixing; if the background keyboard is mixed too loudly or too artificially compared to real call conditions, the model may overfit to the pattern and perform poorly on real microphone input.

7. Testing / Results / Shortcomings

The following sections will highlight the results found during testing, alongside an analysis of what areas were a success and which were not a success.

7.1 Multi-Label Classification

The model is tested on two different metrics: the ability to correctly label different mixed audio clips and the ability to dissect a longer audio file into smaller windows and correctly apply attenuation to the entire file. While the attenuation strategy is hard to quantify, the labelling accuracy is measurable.

7.1.1 Model Testing / Evaluation

The following table shows the accuracy of the model at predicting noise classes and speech.

Test Metrics	**Loss**	**Binary Accuracy**	**Precision**	**Recall**
Values	0.8316	0.851	0.8469	0.8379

Table 2: Accuracy of classification model

The multi-label classification model achieved a binary accuracy of 0.8510, precision of 0.8469, and recall of 0.8379, with a test loss of 0.8316. These results indicate that the model is generally able to distinguish between target noise classes and clean audio with reasonably strong performance. The similar precision and recall values suggest that the model is not heavily biased toward either over-detecting or under-detecting noise, and instead provides a fairly balanced classification result. Overall, this shows that the model is capable of identifying the presence of multiple sound classes in short audio windows with acceptable reliability for the purposes of this project.

7.1.2 Attenuation Results on Longer Audio

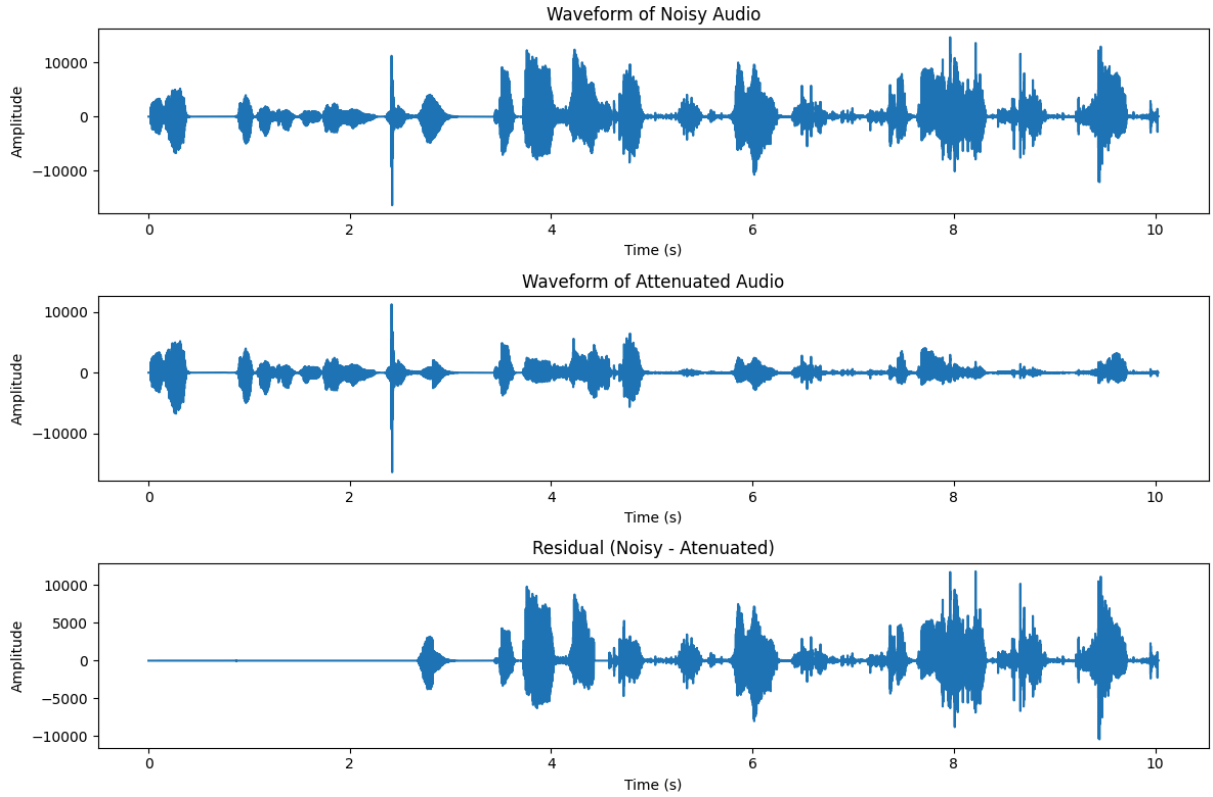

Figure 7: Waveform of Noisy vs. Attenuated Audio

The waveform comparison between the noisy input, denoised output, and residual signal shows that the attenuation system is successfully removing a large portion of the unwanted content. In the denoised waveform, several high-energy regions present in the noisy input are significantly reduced or removed, while lower-energy speech sections are still retained. This suggests that the model is able to identify many noisy windows correctly and apply suppression in a way that reduces overall interference.

At the same time, the denoised waveform also reveals several shortcomings. Some sections are attenuated very aggressively, creating near-silent gaps in the output. This suggests that the system may sometimes suppress not only noise, but also parts of the desired speech signal. Because attenuation is applied on a window basis, the transitions can appear abrupt, especially when a short segment is classified as noisy while surrounding segments are classified as clean. This can make the output sound choppy or unnatural.

Overall, the results demonstrate that this method is effective at reducing noise presence in longer recordings, but its main limitation is performing window-level attenuation rather than true source separation. This makes the system suitable for demonstrating suppression behavior, but less suitable when preservation of speech quality is the highest priority.

7.1.3 Real-Time Testing / Evaluation

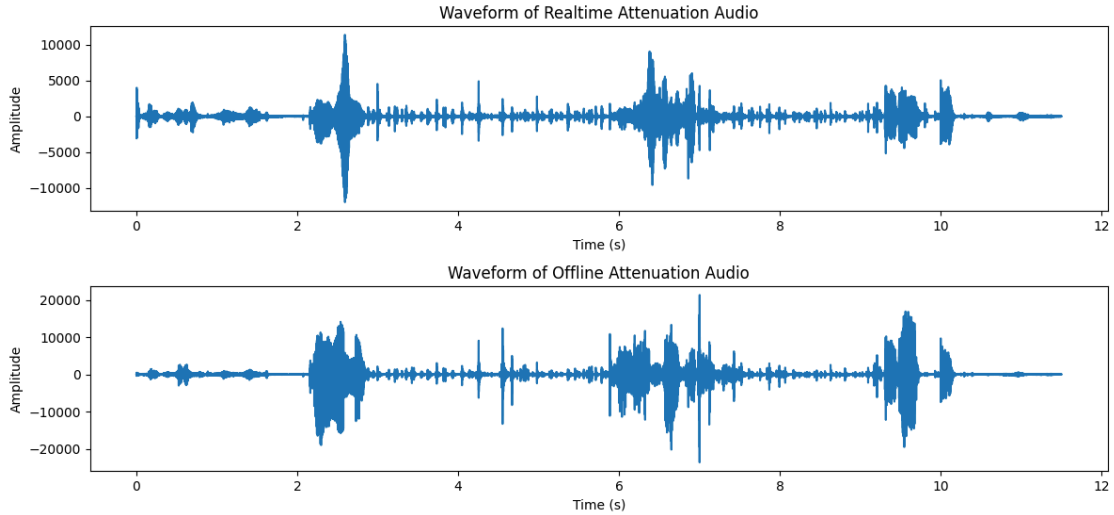

Figure 8: Real-time vs. Offline Attenuation Audio

The waveform for realtime attenuation looks drastically different than storing the realtime audio and doing the attenuation on the entire audio file offline. This difference is mainly caused by the tradeoff between latency and context. In the offline case, the algorithm has access to the complete audio signal and can make attenuation decisions using longer contextual windows over the entire recording. In contrast, the real-time system must make decisions immediately using only the most recent buffered audio, which limits the amount of context available at each step.

In these cases, the short-window predictor may assign a stronger speech-plus-noise confidence than the longer contextual predictor would, which leads to more aggressive attenuation being applied in real time. This explains why the real-time attenuated waveform shows a lower overall amplitude in several regions compared to the offline attenuated waveform.

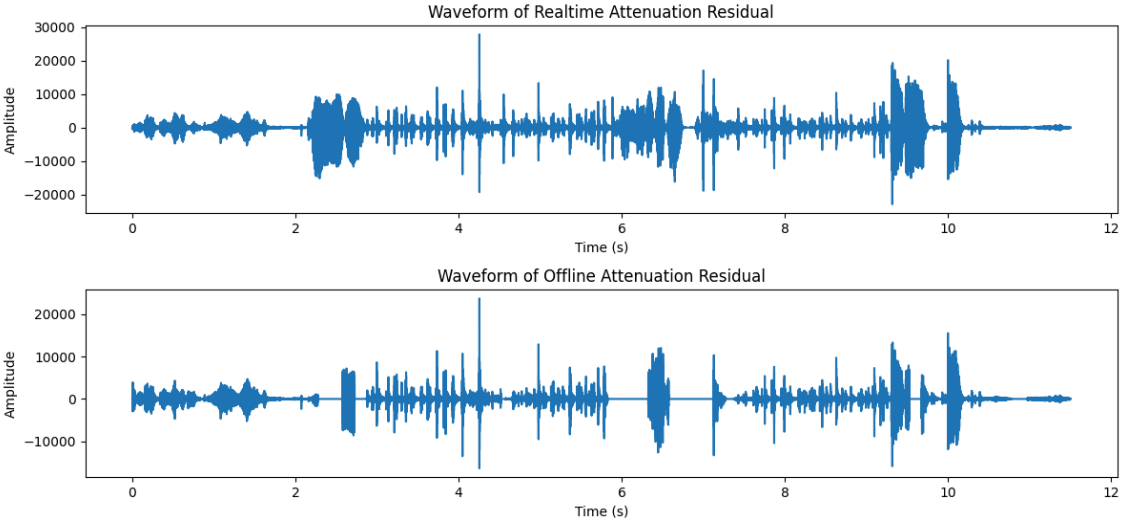

Figure 9: Residual of Real-time vs. Offline Attenuation Audio

The residual graph further proves this theory, that offline attenuation is more accurate than real-time attenuation. However, exceptions are present during segments containing more consistent and sustained noise. In those regions, the real-time and offline attenuation results are more similar. This indicates that the real-time method performs reasonably well when the noise characteristics remain stable over time. The main weakness therefore arises not from steady noise, but from short, ambiguous, or rapidly changing events, where distinguishing speech from transient noise is much more difficult without additional context.

7.2 STFT Masking System Findings

This section will highlight the findings during testing of the STFT-based masking system design. Before continuing, all files used for this section are stored in “test input” and “test output” folders of the code submission for STFT.

7.2.1 Model Testing / Evaluation

For determining the quality of the final output, the model was provided with several test files to determine the quality of the final output based on three parameters: Signal-Noise Ratio (SNR), Short-Time Objective Intelligibility (STOI) and Perceptual Evaluation of Speech Quality (PESQ). SNR measures the ratio of signal to noise in the provided audio, with a higher SNR dB value implying easier time identifying the original audio speech [20]. STOI is a measure of how intelligible the original speech file is compared to post-processing [21]. PESQ measures the audio quality itself in comparison to the original file, looking at features like audio sharpness and the presence of background noise [22].

The 4 audio files of the following table are examples of noisy audio clips post-denoising, compared with the original:

Audio Clips	**SNR (dB)**	**STOI**	**PESQ**
clnsp1.wav	12.22	0.9230	1.4152
clnsp2.wav	10.90	0.8012	1.3548
clnsp5.wav	15.24	0.9123	2.1483
clnsp13.wav	9.82	0.6625	1.3092
Average of first 4	12.05	0.82475	1.556875

Table 3: Audio quality metrics for 4 test audio clips

Observing the results in table 3, the overall results of the system is not optimal to use in practice as of yet. Values to be considered “good” for audio would be as follows: ~20 dB for SNR [20], ~1 for STOI [21] and ~3 for PESQ [22]. When comparing the standards to the values found, the SNR values ranged from approximately 10 to 15 dB, implying audio the strength of the signal is not negligible but still has room to improve. The STOI values were closer to approaching perfect for cases 1 and 3 but still other noise effects still show areas of improvement as they near the 0.7 to 0.8 range. Lastly, the PESQ

values are showing the worst results in comparison to the good standard. Overall, the system can still undergo much development to improve the results and produce more accurate, denoised audio files. The current hypothesis for this outcome lies in two areas: the dataset variety and the masks detection of noise to speech. This could potentially be fixed by growing the dataset and allowing the model to see more variety to combat different variations.

The figure is the waveform for the audio file clnsp5.wav. Figure 10.1 is the waveform of the original audio, figure 10.2 is a display of the speech file mixed with a typing noise file, figure 10.3 is the denoised post inference and figure 10.4 is a display of the residual between the first two waves:

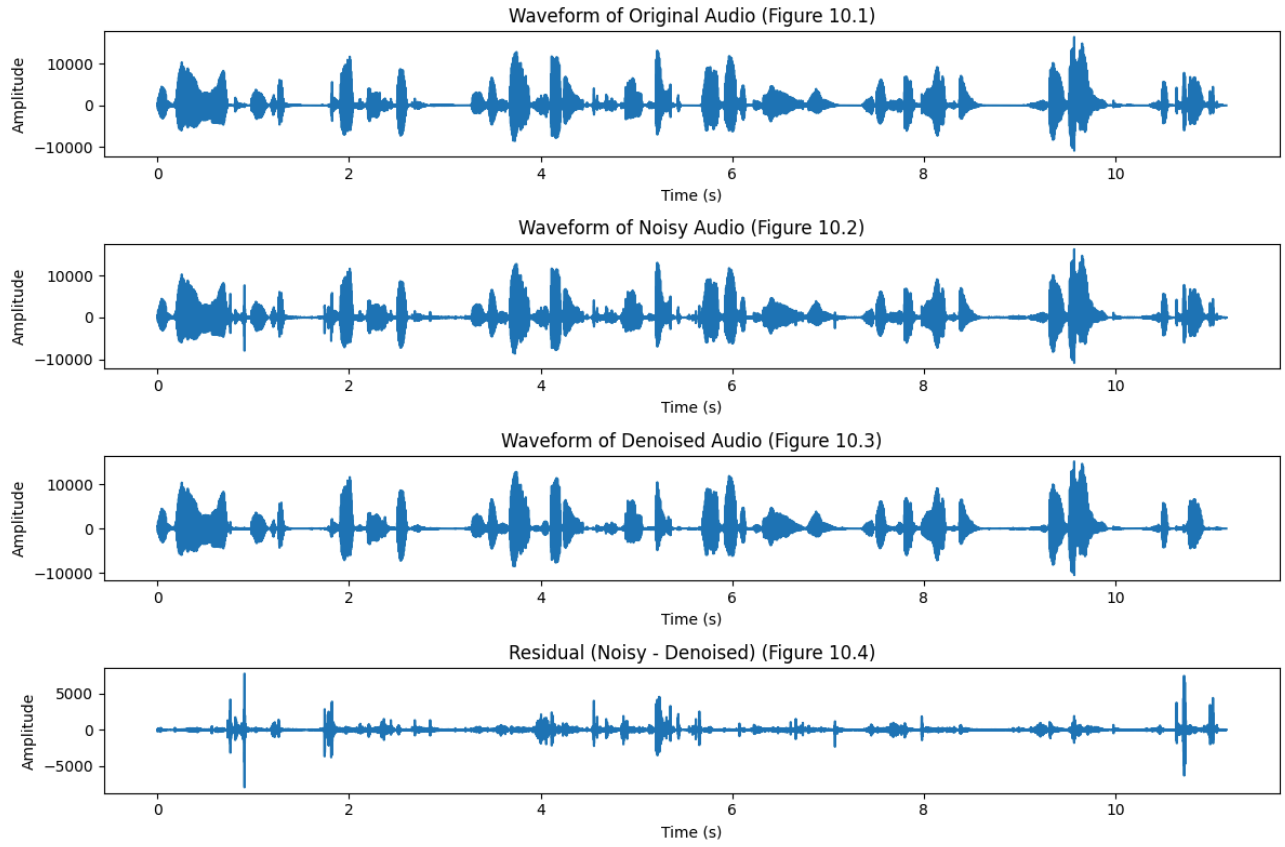

Figure 10: Array of clean, noisy, denoised and residual waveforms for file clnsp5.wav

As seen in the final residual figure, the denoising model was able to filter out high peaks in the audio file, particularly evident at approximately the 1 second and 4.5 second mark. When listening to the audio files themselves, while not all the noise was able to be filtered out, the reduction in volume for the eating sounds were able to be reduced, resulting in an output file with a relatively clear speech. Similar to the previous section regarding metrics, a larger dataset and letting the model understand in more detail should allow a better masking effect to further reduce the noise in the output.

7.2.2 Real-Time Testing / Evaluation

When testing for real-time performance, different block sizes are chosen to test if it affects latency. 3 latency measurements were taken per size which includes the model inference time, the STFT and ISTFT processing time, and the final playback delay experienced by the user due to processing and buffer delays..

Block size	Model (ms)	Transform + Model (ms)	Playback Latency (ms)
1024	3.49	4.45	134.15
512	3.12	4.45	71.57

Table 4: Latency values from real-time implementation (may differ based on hardware)

When comparing with online sources regarding human audio latency perception, the ranges between 20 to 50 ms will result in the end user slightly noticing discrepancies and latency above 100ms resulting in clearly noticeable delay [23]. The project was ultimately able to achieve the lowest system latency with 512 sample size of 71.57 ms, implying that the end user will be able to notice the delay but this should not result in unnatural conversations during usage. Although not real-time in the sense of a 0 ms delay between processing, the group was able to achieve a reasonable final output.

The following figure displays the effects of processing real-time recorded audio files through the offline inference script and processing through the real-time processing script.

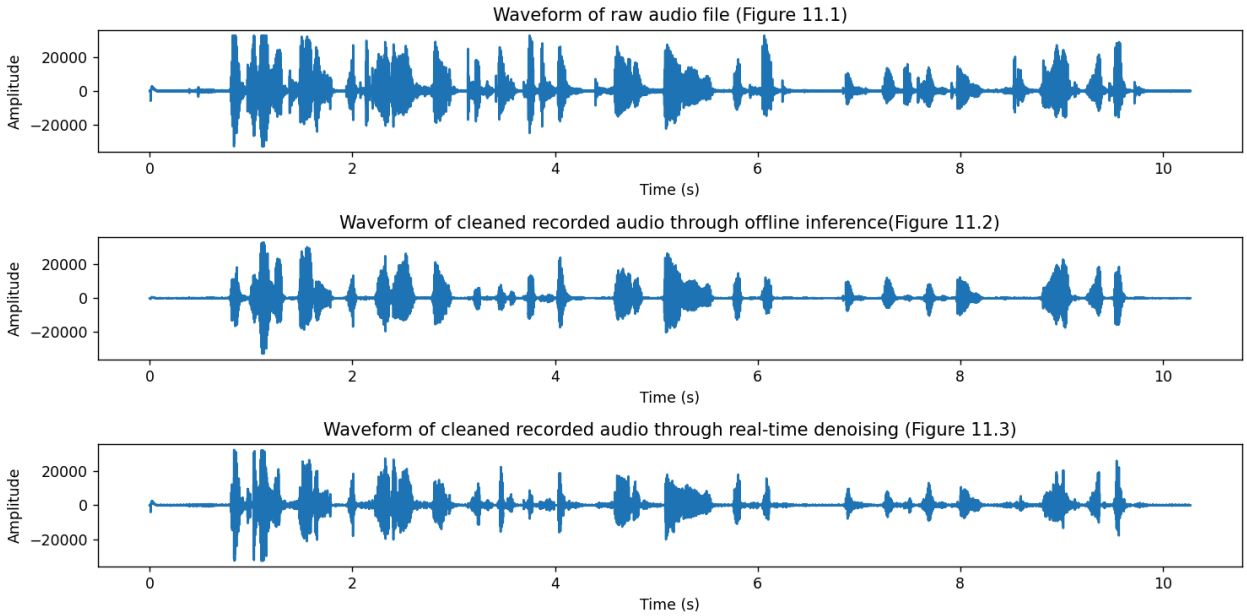

Figure 11: Difference between processing offline vs. real-time

When comparing the figures 11.2 and 11.3, it can be seen that the real-time denoising fails to suppress some noises that are otherwise successfully removed during offline inference. This is due to the

same latency-versus-performance tradeoff observed in the multi-label classification system, where long-term context must be sacrificed in order for the masking to react immediately.

8. Improvements / Future Plans

While the current designs are capable of denoising the audio files at a moderate level, there is much left to attempt in the system. Future plans with the systems aim to improve the audio denoising effectiveness and allow a smoother real-time experience with less stuttering effects in the final output. This is hoped to be achieved with further trials and testing to reach audio capabilities similar to large scale applications.

Once the base design is further redefined, the project will be adapted to be used with a microcontroller in a lightweight configuration to allow a more plausible real-time usage without the use of a laptop or desktop to process the information. This may possibly grow to allow more features than simply audio denoising like a portable audio translation system on the microcontroller.

9. Comparison to Initial Plans

Compared to the initial plans made during the proposal, many changes had to be made to the initial schedule to account for each member's schedules and for investing time in debugging. The initial planning had allocated relatively equal amounts of time between making an appropriate final design and debugging the design, but there were more problems than anticipated in regards to debugging the design and audio output audio files not masking the audio as intended. Certain instances would manage to add undesired audio effects to the final outputs rather than masking the noise or the real-time system would give a headache when attempting to achieve an output with less stuttering effects. Ultimately, while a more robust system would have been desired, the project was an overall success in teaching the group the effectiveness in attempting different approaches when designing AI systems and learning to optimize the system for each use case. A lot of help was found through coding resources like PyTorch and TensorFlow official websites for documentation [24][25].

10. Conclusion

While the current project still has many shortcomings in terms of performance, the core objectives are considered to have been achieved. The group was able to successfully create a system that was able to filter/reduce the background noise during speech and was also able to learn a lot through the process of trial and error through making a better performing system. The group hopes to further develop the design and improve the performance for future personal projects.

11. Bibliography

[1] "How Microsoft Teams uses AI to enhance audio and video in meetings - Microsoft Support," *support.microsoft.com*.

https://support.microsoft.com/en-gb/office/how-microsoft-teams-uses-ai-to-enhance-audio-and-video-in-meetings-40e054ef-2b7a-4b19-9bd0-e7cd3288a5a6
[2] "World's #1 Noise Cancelling App that cancels background noise and echo in meetings.," *Krisp*. https://krisp.ai/noise-cancellation/
[3] "What is behind Krisp Technology?," *Krisp Help*, 2025. https://help.krisp.ai/hc/en-us/articles/360012035480-What-is-behind-Krisp-Technology (accessed Mar. 18, 2026).
[4] immohann, "GitHub - immohann/Denoiser: An AI model to remove noise from the input audio using deep learning model which predicts the type of noise present and filter it out from the audio to give noise-free results.," *GitHub*, 2025. https://github.com/immohann/Denoiser/tree/master (accessed Mar. 19, 2026).
[5] T. S. Silva, "Practical Deep Learning Audio Denoising - Thalles' blog," *Github.io*, Dec. 18, 2019. https://sthalles.github.io/practical-deep-learning-audio-denoising/ (accessed Mar. 19, 2026).
[6] J. S. Ashwin and N. Manoharan, "Audio Denoising Based on Short Time Fourier Transform," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 9, no. 1, p. 89, Jan. 2018, doi: https://doi.org/10.11591/ijeecs.v9.i1.pp89-92.
[7] Microsoft, "microsoft/MS-SNSD," *GitHub*, Mar. 20, 2024. https://github.com/microsoft/MS-SNSD (accessed Mar. 20, 2026).
[8] "AudioSet," *Google.com*, 2026. https://research.google.com/audioset/dataset/speech.html (accessed Mar. 20, 2026).
[9] J. S. Ma, "Eating Sound Collection," *Kaggle.com*, 2020. https://www.kaggle.com/datasets/mashijie/eating-sound-collection (accessed Mar. 21, 2026).
[10] karolpiczak, "ESC-50/meta/esc50.csv at master · karolpiczak/ESC-50," *GitHub*, 2025. https://github.com/karolpiczak/ESC-50/blob/master/meta/esc50.csv (accessed Mar. 21, 2026).
[11] P. Pinto, Rafael, da, and S. Malta, "Audio dataset of keyboard keystrokes," *Zenodo*, Nov. 2025, doi: https://doi.org/10.5281/zenodo.17608997.
[12] mariostrbac, "GitHub - mariostrbac/environmental-sound-classification: Environmental sound classification with Convolutional neural networks and the UrbanSound8K dataset.," *GitHub*, 2025. https://github.com/mariostrbac/environmental-sound-classification (accessed Mar. 24, 2026).
[13] W. Mu, B. Yin, X. Huang, J. Xu, and Z. Du, "Environmental sound classification using temporal-frequency attention based convolutional neural network," *Scientific Reports*, vol. 11, no. 1, Nov. 2021, doi: https://doi.org/10.1038/s41598-021-01045-4.
[14] "tf.nn.weighted_cross_entropy_with_logits | TensorFlow v2.12.0," *TensorFlow*. https://www.tensorflow.org/api_docs/python/tf/nn/weighted_cross_entropy_with_logits (accessed Mar. 25, 2026).
[15] "Short-Time Fourier Transforms." https://course.ece.cmu.edu/~ece491/lectures/L25/STFT_Notes_ADSP.pdf (accessed Mar. 23, 2026).
[16] "Denoise Speech Using Deep Learning Networks - MATLAB & Simulink," *www.mathworks.com*. https://www.mathworks.com/help/deeplearning/ug/denoise-speech-using-deep-learning-networks.html (accessed Mar. 23, 2026).
[17] L.-D. Jimon and M.-F. Vaida, "AUDIO DENOISING USING U-NET ARCHITECTURE," vol. 65, no. 1, Nov. 2025, Accessed: Mar. 25, 2026. [Online]. Available: https://users.utcluj.ro/~atn/papers/ATN_1_2025_2.pdf

[18] “SoundCard — SoundCard 0.2.0 documentation,” *Readthedocs.io*, 2016. https://soundcard.readthedocs.io/en/latest/ (accessed Apr. 01, 2026).
[19] SciPy, “SciPy.org,” *Scipy.org*, 2020. https://scipy.org/ (accessed Apr. 01, 2026).
[20] J. Semmlow, “The Big Picture,” *Signals and Systems for Bioengineers*, pp. 3–33, 2012, doi: https://doi.org/10.1016/b978-0-12-384982-3.00001-8.
[21] “Short-Time Objective Intelligibility (STOI) — PyTorch-Metrics 1.3.1 documentation,” *lightning.ai*. https://lightning.ai/docs/torchmetrics/stable/audio/short_time_objective_intelligibility.html (accessed Apr. 03, 2026).
[22] “Perceptual Evaluation of Speech Quality (PESQ) — PyTorch-Metrics 1.3.2 documentation,” *lightning.ai*. https://lightning.ai/docs/torchmetrics/stable/audio/perceptual_evaluation_speech_quality.html (accessed Apr. 03, 2026).
[23] Cora, “What Is Audio Latency? Meaning, Causes, Measurement & How to Reduce It,” *Sponcomm.com*, Feb. 06, 2026. https://sponcomm.com/info-detail/audio-latency (accessed Apr. 04, 2026).
[24] PyTorch, “PyTorch documentation — PyTorch 2.7 documentation,” *Pytorch.org*, 2024. https://docs.pytorch.org/docs/stable/index.html (accessed Mar. 10, 2026).
[25] TensorFlow, “API documentation | tensorflow core v2.4.1,” *TensorFlow*, Sep. 30, 2024. https://www.tensorflow.org/api_docs (accessed Mar. 10, 2026).